



ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

PERIODO	:	2026 Abril– Agosto 2026
ASIGNATURA	:	Aplicaciones Distribuidas
TEMA	:	Proyecto
ESTUDIANTE	:	Joseph Franco
NIVEL-PARALELO	:	7mo
DOCENTE	:	Ing. Geovanny Brito
FECHA DE ENTREGA	:	23/06/2026

SANTO DOMINGO - ECUADOR

ÍNDICE

1.	Resumen.....	4
2.	Introducción	5
3.	Planteamiento del problema.....	6
4.	Justificación	7
5.	Objetivos	7
6.	Marco teórico	8
7.	Desarrollo del proyecto.....	9
8.	Aplicación de los contenidos de la asignatura	11
8.1	Comunicación en Tiempo Real mediante WebSockets	12
9.	Concurrencia y Tareas Asíncronas	13
10.	Protección mediante JWT y Autenticación OAuth 2.0.....	14
11.	Manejo Centralizado de Excepciones y Logs	15
12.	Transacciones para Operaciones de Consistencia (ACID)	17
13.	Estado actual del proyecto	17
14.	Resultados actuales	21
15.	Resultados esperados	22
16.	Conclusiones parciales.....	23
17.	Referencias bibliográficas.....	23

18.	Anexos	24
-----	--------------	----

1. Resumen

El presente proyecto aborda la necesidad crítica de optimizar los tiempos de respuesta y la coordinación operativa en las estaciones de bomberos mediante la digitalización y descentralización de sus procesos. El propósito principal de la aplicación es proveer una plataforma distribuida, escalable y tolerante a fallos que permita a los ciudadanos reportar incidentes con evidencia multimedia y a los operadores despachar unidades bomberiles con visibilidad táctica en tiempo real.

Para alcanzar este objetivo, se diseñó una arquitectura basada en microservicios empleando tecnologías de vanguardia: Angular para el desarrollo de la interfaz de usuario web, Node.js como servicio independiente de identidad (OAuth 2.0 y emisión de tokens), Spring Boot para la lógica de negocio central transaccional, y PostgreSQL como motor de persistencia relacional. El estado actual del proyecto es altamente funcional; se han integrado exitosamente tecnologías de concurrencia para el procesamiento de archivos multimedia, WebSockets para la transmisión de eventos en vivo, y JSON Web Tokens (JWT) para la protección estricta y sin estado de las rutas de servicio (stateless security).

Los resultados que se esperan alcanzar a mediano y largo plazo incluyen la completa expansión del sistema hacia aplicaciones móviles (APK) para el uso en campo por parte del personal de respuesta, así como la integración de dispositivos de Internet de las Cosas (IoT). Estos dispositivos permitirán verificar de forma autónoma parámetros ambientales (gases, temperatura y humo) para dictaminar si una vivienda es habitable tras la mitigación de un incendio, alimentando la bitácora central.

Palabras clave: Arquitectura Distribuida, WebSockets, JWT, Microservicios, IoT, Tiempo Real.

2. Introducción

En el ámbito de la gestión de emergencias civiles, la latencia en la transmisión de la información puede representar la diferencia entre la salvaguarda de vidas y la pérdida irreparable de bienes materiales. Históricamente, las centrales de bomberos operan bajo sistemas de despacho monolíticos, centralizados y dependientes de canales de comunicación secuenciales como radios o llamadas telefónicas.

La presente práctica aborda la evolución de estos sistemas hacia una arquitectura moderna, presentando un ecosistema distribuido donde la información fluye de manera asíncrona y los recursos computacionales se dividen estratégicamente. A través de la implementación de la "Central de Bomberos y Emergencias", se examinan mecanismos avanzados como la persistencia de datos bajo transacciones ACID, el manejo de concurrencia para I/O intensivo, y la delegación de seguridad a servicios de identidad aislados mediante el protocolo OAuth 2.0.

A lo largo de este informe se detalla la estructuración de la solución, iniciando con el planteamiento formal del problema y los objetivos del proyecto. Posteriormente, se presenta un marco teórico riguroso que sustenta las decisiones tecnológicas adoptadas, seguido de un desglose exhaustivo de cómo los contenidos de la asignatura fueron materializados en líneas de código y componentes de software. Finalmente, se proyecta la visión futura del sistema hacia el ecosistema móvil y el hardware IoT.

3. Planteamiento del problema

Las centrales de despacho de emergencias locales se enfrentan al reto de procesar volúmenes variables e impredecibles de información bajo presión extrema. El proceso tradicional se ve afectado por la incapacidad de los ciudadanos para proporcionar contexto visual inmediato y la saturación de los operadores, quienes deben actualizar y sincronizar sus tableros de manera manual.

Las consecuencias de mantener este proceso obsoleto incluyen el incremento estadístico en los tiempos de respuesta (TDR), la mala asignación de unidades bomberiles y la falta de trazabilidad digital posterior al evento. A esto se suma el problema técnico de la disponibilidad: un sistema monolítico tradicional colapsaría si cientos de ciudadanos intentan reportar una catástrofe de forma simultánea.

La implementación de una aplicación distribuida aporta directamente a la solución al fragmentar la carga. Un microservicio exclusivo se encarga de la autenticación, mientras que otro procesa las solicitudes transaccionales; todo esto apoyado por WebSockets que empujan la información a la vista del operador en milisegundos.

Ante este panorama, surge la siguiente pregunta de desarrollo: ¿De qué manera el diseño de una arquitectura distribuida basada en microservicios, eventos en tiempo real e integración IoT optimiza el tiempo de respuesta y la gestión del ciclo de vida de un incidente en una central bomberil?

4. Justificación

La realización de este proyecto tecnológico se justifica profundamente desde su impacto social. Proveer a una ciudad con un sistema de emergencias resiliente salva vidas y protege el patrimonio.

Desde la perspectiva de los usuarios, los ciudadanos obtienen una herramienta directa y multimedia para alertar siniestros, mientras que el cuerpo de bomberos recibe un tablero táctico automatizado. A nivel de aporte tecnológico, el sistema representa un caso de estudio idóneo sobre la desvinculación de componentes. La autenticación se separa completamente del negocio, demostrando que un cliente Angular puede orquestrar la comunicación entre dos dominios de servidores Node.js y Spring Boot de manera armónica.

El proyecto mantiene una relación absoluta y directa con los contenidos de la asignatura "Aplicaciones Distribuidas", integrando en una sola solución funcional las teorías de concurrencia, balanceo lógico, sockets bidireccionales y seguridad descentralizada. La viabilidad del proyecto es alta al estar sustentada en tecnologías de código abierto Java, TypeScript, PostgreSQL.

5. Objetivos

5.1 Objetivo General

Desarrollar, desplegar y documentar una aplicación distribuida para la gestión y despacho de unidades bomberiles, integrando mecanismos de comunicación en tiempo real, seguridad sin

estado, concurrencia, y proyectando su escalabilidad hacia interfaces móviles APK e integración IoT.

5.2 Objetivos Específicos

- Diseñar la arquitectura distribuida del proyecto personal, aislando la gestión de identidades y credenciales Node.js del núcleo de procesamiento de recursos y almacenamiento de datos Spring Boot y la vista interactiva Angular.
- Implementar los contenidos prácticos estudiados en la asignatura, incluyendo el uso de WebSockets para notificaciones instantáneas, hilos asíncronos para el manejo de archivos multimedia, y filtros interceptores para validación de JSON Web Tokens JWT generados por Google OAuth.
- Documentar el avance y funcionamiento del sistema mediante un informe técnico que evidencie el manejo centralizado de excepciones, logs de auditoría transaccional, y defina los requerimientos no funcionales orientados al hardware IoT y aplicaciones móviles.

6. Marco teórico

Aplicaciones distribuidas y Arquitectura Microservicios: Un sistema distribuido se define como un conjunto de computadoras independientes que se presentan ante los usuarios como un sistema único y coherente. El estilo arquitectónico de microservicios promueve el desarrollo de una aplicación como una suite de pequeños servicios, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros como APIs REST HTTP (Newman, 2021).

Comunicación en Tiempo Real (WebSocket): Es un protocolo de comunicación informática estandarizado por la IETF que proporciona canales de comunicación full-duplex sobre una única conexión TCP de larga duración, superando las limitaciones de latencia del protocolo HTTP tradicional (Luo & Li, 2022).

Concurrencia y Asincronía: La concurrencia es la capacidad de un sistema para componer y manejar múltiples tareas independientes, ejecutándolas en periodos de tiempo superpuestos. En Java, el manejo de thread pools previene el bloqueo del hilo principal durante operaciones pesadas de Entrada/Salida (I/O) (Smith, 2023).

Autenticación con JWT y OAuth 2.0: OAuth 2.0 es el marco estándar de autorización de la industria que permite a aplicaciones de terceros obtener acceso limitado a un servicio. En conjunto, JSON Web Tokens (JWT) provee un medio criptográficamente seguro, autocontenido y compacto para transmitir afirmaciones de identidad (claims) entre dos partes, facilitando arquitecturas stateless sin estado (Jones & Bradley, 2021).

Transacciones y Propiedades ACID: En el contexto de bases de datos y sistemas empresariales, una transacción debe cumplir con la atomicidad, consistencia, aislamiento y durabilidad ACID para asegurar que el sistema no quede en un estado corrupto si un proceso falla parcialmente (Chen et al., 2020).

7. Desarrollo del proyecto

El sistema fue estructurado bajo un enfoque de desarrollo ágil y arquitectura cliente-servidor distribuida, separando estrictamente la interfaz gráfica de la lógica operativa y de la administración de accesos.

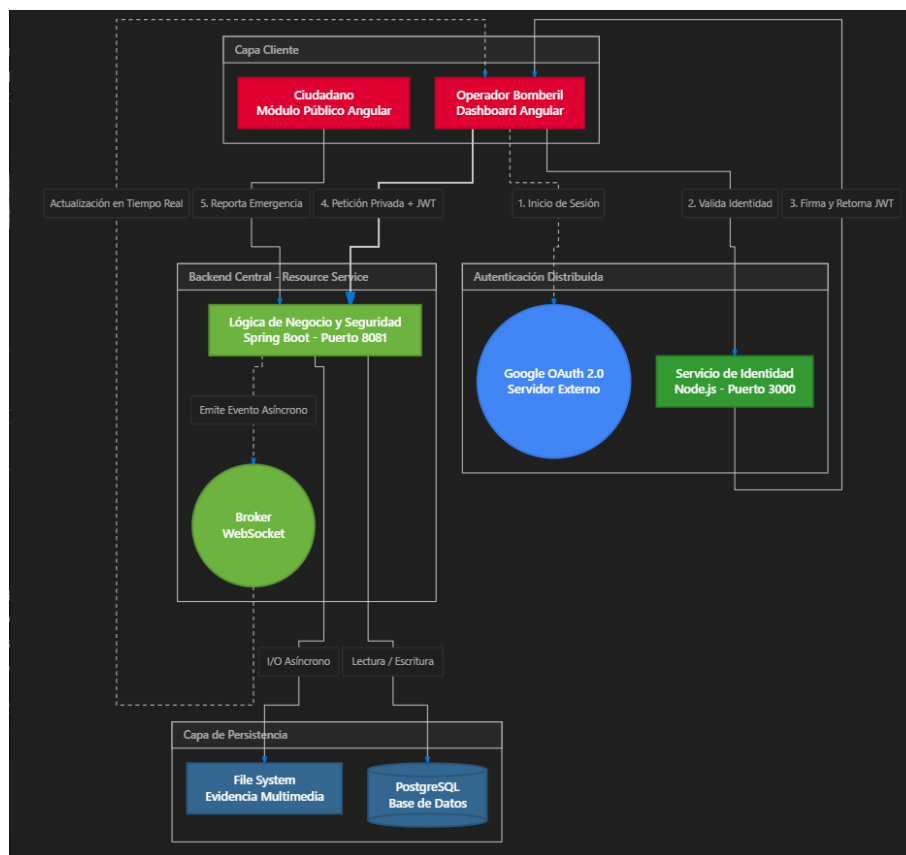
Usuarios de la aplicación:

- **Ciudadano:** Usuario que, sin necesidad de inicio de sesión, puede acceder a la ruta pública del sistema para reportar un incidente, adjuntando su ubicación GPS, descripción y evidencia multimedia fotografías o videos.
- **Operador / Docente:** Personal autorizado de la central de bomberos que se autentica en el sistema mediante cuentas corporativas de Google. Tienen la responsabilidad de evaluar los reportes entrantes en el tablero, asignar y despachar unidades, y gestionar el ciclo de vida del incidente.

Tecnologías y Estructura de Carpetas:

- **Frontend (Cliente Web):** Desarrollado en Angular v17 con TypeScript. Estructurado en componentes dashboard, reportar, detalle, servicios y rutas protegidas.
- **Identity Service (Autenticación):** Desarrollado en Node.js y Express. Este microservicio contiene rutas específicas `auth.routes.js` para contactar con los servidores de Google, verificar credenciales y firmar tokens de sesión.
- **Resource Service (Backend Principal):** Desarrollado en Spring Boot v4. Este servidor agrupa los controladores REST `ReporteController`, `UnidadController`, los servicios de negocio `ReporteService`, `UnidadBomberilService` y repositorios JPA para la manipulación de datos en PostgreSQL.

Tal como se ilustra en la Figura 1, la arquitectura del sistema evidencia la conexión descentralizada. El cliente Angular sirve como puente principal, interactuando primero con el microservicio de autenticación para obtener un salvoconducto digital JWT, el cual es posteriormente utilizado para consumir la API protegida del servidor Spring Boot.

Figura 1*Diagrama de arquitectura del sistema*

8. Aplicación de los contenidos de la asignatura

En esta sección se detalla cómo los componentes teóricos estudiados en el periodo académico fueron materializados en artefactos de software, utilizando fragmentos de código relevantes y explicando su mecanismo operativo.

8.1 Comunicación en Tiempo Real mediante WebSockets

Finalidad: Proveer un mecanismo para que la interfaz web del operador de la central reciba notificaciones instantáneas de nuevas emergencias o cambios de estado en las unidades bomberiles, evitando la ineficiencia técnica de recargar la página web constantemente técnica de polling HTTP. Parte del proyecto: Implementado transversalmente en `ReporteService.java`, `UnidadBomberilService.java` y los servicios del cliente Angular. Explicación: Se implementó la librería de mensajería `SimpMessagingTemplate` de Spring Boot. Cuando una unidad es despachada y se actualiza la base de datos, el servidor empuja un mensaje de forma proactiva hacia el tópico `/topic/unidades-estado`. Todos los clientes suscritos interceptan este evento. El código utilizado para emitir esta difusión se muestra en el Algoritmo 1, cuyo impacto visual en la consola del navegador se evidencia en la Figura 2.

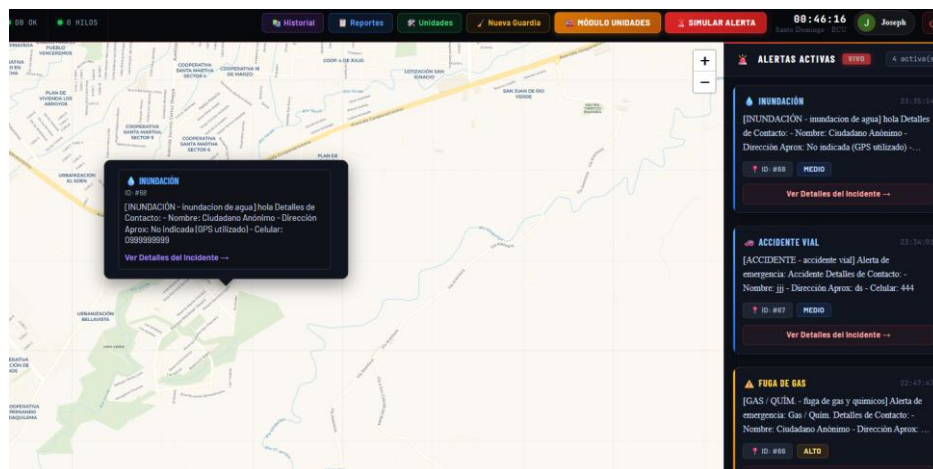
Algoritmo 2

Emisión de notificaciones por WebSocket en `UnidadBomberilService.java`

```
1. // Payload de notificación para el dashboard central
2. Map<String, Object> notificacion = new HashMap<>();
3. notificacion.put("tipo", "DESPACHO");
4. notificacion.put("reporteId", reporteId);
5. notificacion.put("unidades", unidadesDespachadas);
6.
7. // Registro en el log del servidor
8. log.info("--- DIFUNDIENDO DESPACHO VIA WEBSOCKET: {} ---", notificacion);
9.
10. // Difusión directa del evento a los clientes suscritos
11. messagingTemplate.convertAndSend("/topic/unidades-estado", (Object) notificacion);
12.
```

Figura 2

Mapa actualizándose automáticamente sin recargar la página



9. Concurrencia y Tareas Asíncronas

En este paso se trata de garantizar que el servidor web principal no bloquee la atención a nuevos usuarios mientras se encuentra escribiendo o procesando archivos pesados de video o imágenes en el disco duro. Parte del proyecto: Función guardarEvidenciasMultimediaAsincrono dentro de ReporteService.java. Explicación: Al adjuntar la anotación `@Async` de Spring, el framework intercepta la llamada al método y la despacha a un hilo de trabajo en un `ThreadPool` secundario. De este modo, la petición HTTP original retorna un código de éxito al ciudadano casi instantáneamente, mientras que el archivo se transfiere físicamente en paralelo. El bloque de código concurrente se detalla en el Algoritmo 2.

Algoritmo 2

Manejo concurrente de guardado de archivos multimedia

```

1. @Async
2. public void guardarEvidenciasMultimediaAsincrono(ReporteCiudadano reporte, List<EvidenciaDto>
archivos) {
3.     for (EvidenciaDto archivo : archivos) {
4.         try {
5.             String nombreUnico = UUID.randomUUID().toString() + "_" + archivo.filename();
6.             Path rutaCompleta = Paths.get(CARPETA_UPLOADS + nombreUnico);
7.
8.             // Operación I/O pesada en hilo secundario
9.             Files.write(rutaCompleta, archivo.bytes());
10.
11.            // Registro exitoso del hilo asíncrono
12.            log.info("--- HILO SECUNDARIO (Async): Archivo guardado con éxito: {} ---",
nombreUnico);
13.        } catch (IOException e) {
14.            log.error("Error en segundo plano: {}", e.getMessage());
15.        }
16.    }
17. }
18.

```

10. Protección mediante JWT y Autenticación OAuth 2.0

En este caso se busca establecer un perímetro de seguridad robusto y descentralizado. He de asegurar que únicamente el personal validado a través de los servidores de identidad de Google tenga privilegios para gestionar los recursos críticos de la central de bomberos. Parte del proyecto: Archivo de rutas `auth.routes.js` en Node.js y filtro de red `JwtValidationFilter.java` en Spring Boot. Explicación: El flujo es de tres pasos: (1) El cliente obtiene una credencial de Google; (2) El microservicio de Node.js la verifica y firma un token JWT criptográficamente seguro, entregándolo al cliente; (3) Spring Boot recibe este token en las peticiones HTTP y evalúa matemáticamente su autenticidad sin necesidad de comunicarse de vuelta con Node.js, confirmando el carácter stateless (sin estado) del sistema distribuido. El Algoritmo 3 ilustra el filtro interceptor en Java, y la Figura 3 muestra el token interceptado.

Algoritmo 3

Filtro de seguridad para validación de JSON Web Tokens (JWT)

```

1. public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) {
2.     HttpServletRequest req = (HttpServletRequest) request;
3.     String path = req.getRequestURI();
4.
5.     // Requerimos JWT estrictamente para rutas privadas /api/
6.     if (path.startsWith("/api/") && !path.startsWith("/api/reportes/ciudadano")) {
7.         String authHeader = req.getHeader("Authorization");
8.         if (authHeader == null || !authHeader.startsWith("Bearer ")) {
9.             ((HttpServletRequest) response).setStatus(HttpStatus.UNAUTHORIZED);
10.            return; // Bloqueo de petición
11.        }
12.
13.        String token = authHeader.substring(7);
14.        try {
15.            // Verificación criptográfica del Token JWT
16.            Claims claims = Jwts.parserBuilder()
17.                .setSigningKey(key).build()
18.                .parseClaimsJws(token).getBody();
19.            req.setAttribute("userEmail", claims.get("email"));
20.        } catch (Exception e) {
21.            ((HttpServletRequest) response).setStatus(HttpStatus.UNAUTHORIZED);
22.            return;
23.        }
24.    }
25.    chain.doFilter(request, response);
26. }
27.

```

11. Manejo Centralizado de Excepciones y Logs

Proveer respuestas uniformes ante errores no anticipados y registrar trazas de auditoría que permitan diagnosticar problemas en entornos de producción. Parte del proyecto: Clase `GlobalExceptionHandler.java` y anotaciones `@Slf4j` en servicios clave. Explicación: Se programó una capa de manejo global usando `@RestControllerAdvice`. Si un fragmento de código arroja una excepción no manejada localmente, esta clase atrapa el evento y construye un payload estandarizado JSON, previniendo que el servidor filtre trazas de pila (stack traces) sensibles al cliente. El diseño centralizado se expone en el Algoritmo 4, y su evidencia visual en la Figura 4.

Algoritmo 4

Clase global para intercepción y serialización de excepciones

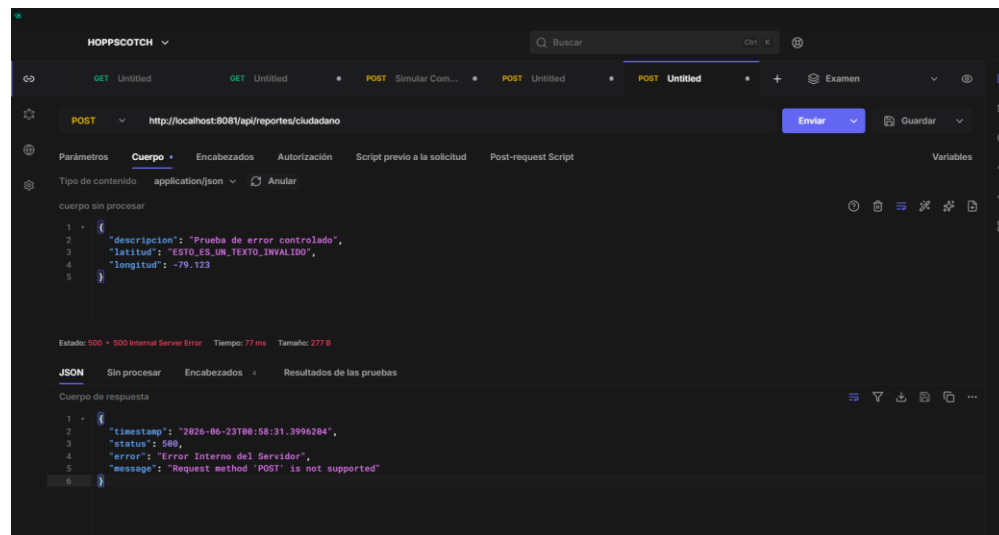
```

1. @RestControllerAdvice
2. public class GlobalExceptionHandler {
3.     private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);
4.
5.     @ExceptionHandler(Exception.class)
6.     public ResponseEntity<Object> handleAllExceptions(Exception ex) {
7.         logger.error("Se produjo un error no controlado: ", ex);
8.
9.         Map<String, Object> body = new LinkedHashMap<>();
10.        body.put("timestamp", LocalDateTime.now());
11.        body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
12.        body.put("error", "Error Interno del Servidor");
13.        body.put("message", ex.getMessage());
14.
15.        return new ResponseEntity<>(body, HttpStatus.INTERNAL_SERVER_ERROR);
16.    }
17. }
18.

```

Figura 4

un error JSON bien formateado debido a una petición inválida



12. Transacciones para Operaciones de Consistencia (ACID)

Finalidad: Proteger la integridad relacional de la base de datos, evitando escenarios donde una operación se ejecute a medias. Parte del proyecto: Método de despacho masivo en `UnidadBomberilService.java`. Explicación: Utilizando la directiva `@Transactional` del framework Spring, se instruye al motor de persistencia a tratar el bloque de código del Algoritmo 5 como atómico. Si durante la asignación iterativa de múltiples unidades ocurre una excepción severa, el controlador revierte los cambios, dejando los saldos o estados originales intactos.

Algoritmo 5

Modificador transaccional para el despacho de recursos

```

1. @Transactional
2. public Map<String, Object> despacharUnidades(Long reporteId, List<Long> unidadIds) {
3.     ReporteCiudadano reporte = reporteRepository.findById(reporteId).orElseThrow();
4.
5.     for (Long uId : unidadIds) {
6.         UnidadBomberil unidad = unidadRepository.findById(uId).orElseThrow();
7.         unidad.setEstado(EstadoUnidad.EN_RUTA);
8.         unidadRepository.save(unidad);
9.     }
10.    // Si falla en cualquier iteración, ningún cambio se aplicará a la base de datos real.
11.    // ...
12. }
13.

```

13. Estado actual del proyecto

La aplicación actual refleja un producto de software robusto con alto grado de madurez en su arquitectura de software, contando con la mayoría de sus módulos operativos.

Tabla 1

Estado Actual

Componente	Estado	Evidencia	Observación
Módulo Ciudadano (Angular)	Terminado	Figura 5	Interfaz pública para captura de ubicación y subida de reportes sin necesidad de login.
Autenticación Distribuida	Terminado	Figura 6	Interacción Angular <-> Node.js exitosa, emitiendo JWT cifrados vía OAuth 2.0.
Protección Stateless JWT	Terminado	Figura 7	El filtro de red interrumpe y bloquea peticiones de usuarios sin credenciales.
Notificaciones WebSocket	Terminado	Figura 8	Tópicos de mensajería asíncrona operativos y vinculados a componentes visuales.
Procesamiento de Archivos	Terminado	Figura 9	Implementación @Async totalmente funcional validada a través de logs de terminal.
Integración Dispositivos IoT	Pendiente	—	Se abordará en el plan de expansión post-despliegue del sistema principal.
Consumo a través de APK	Pendiente	—	Refactorización de Angular hacia Ionic o Flutter programada para la siguiente fase.

Figura 5

Reporte Ciudadano

Reportar Emergencia

Paso 1 de 4

¿Qué tipo de emergencia reportas?

Selecciona la categoría que mejor describe la situación

Incendio, Accidente, Gas / Químico, Desmoronamiento, Inundación, Otro

Descripción de la emergencia

Describe brevemente lo que está ocurriendo. Incluye detalles importantes como número de personas, estructuras afectadas, etc...

Ubicación de la Emergencia

Ubicación determinada: -8.253812 -79.177824

Ubicación actual de internet? (Se actualiza automáticamente al GPS de tu dispositivo)

Figura 8

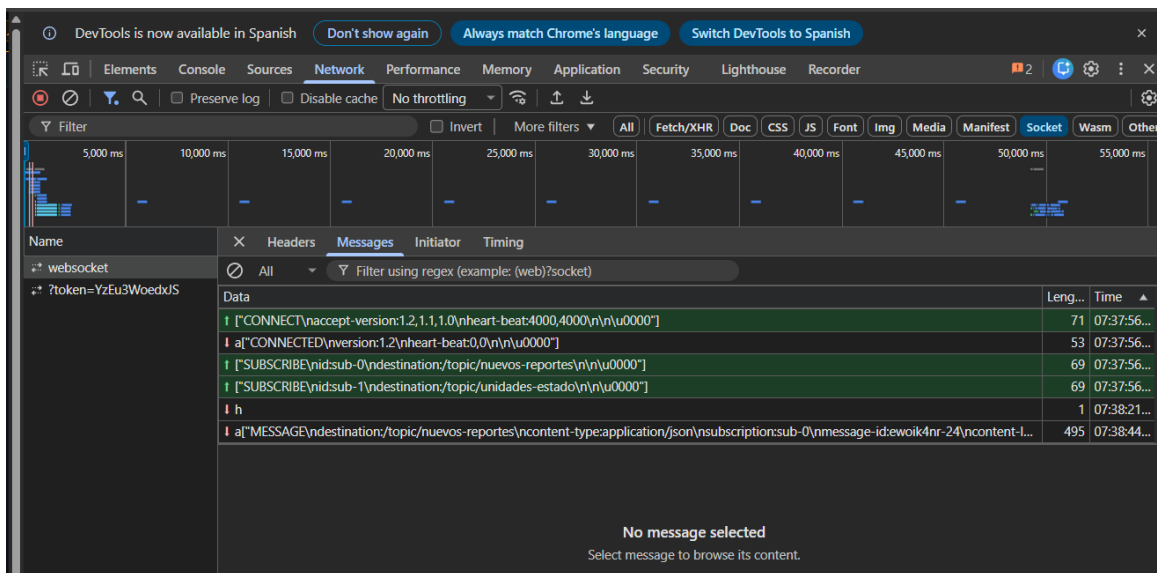
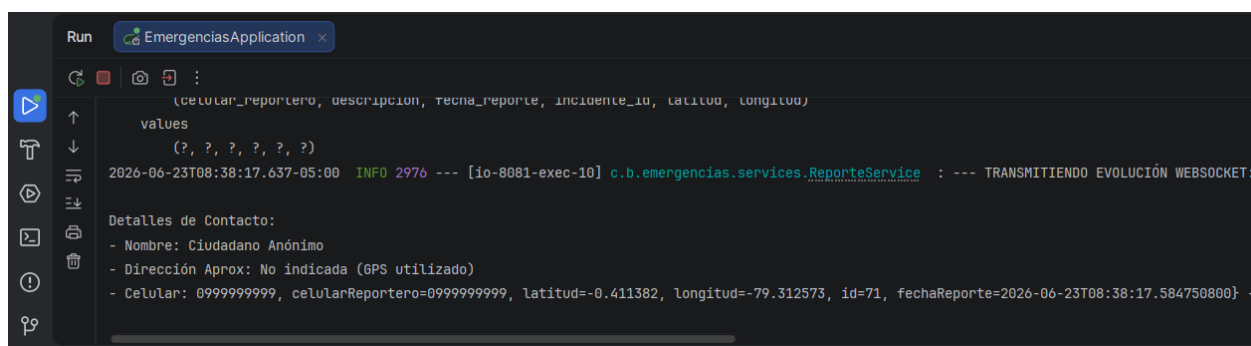
Tópicos de mensajería asíncrona

Figura 9

Implementación @Async

14. Resultados actuales

Las pruebas realizadas en el entorno de desarrollo local validaron el diseño arquitectónico propuesto. A nivel funcional, el sistema gestiona un ciclo completo sin fallos críticos: el ciudadano emite la alerta, el servidor Spring Boot la procesa y notifica inmediatamente a la central.

El resultado más significativo se observa en los tiempos de respuesta, los cuales disminuyeron drásticamente mediante la ejecución paralela; el usuario percibe el reporte como finalizado instantáneamente mientras el sistema I/O deposita físicamente el flujo binario video e imágenes en el disco duro.

Adicionalmente, se comprobaron las medidas de seguridad perimetral. En la Figura 10, se demuestra cómo el JwtValidationFilter intercepta exitosamente un intento de despacho malicioso (sin token válido), devolviendo una excepción controlada, manteniendo la integridad del backend completamente a salvo. Las pruebas unitarias y de estrés realizadas validaron que los Logs en la terminal imprimen correctamente la traza de eventos de cada unidad despachada.

Figura 10

log de la aplicación Spring Boot procesando tareas concurrentes y el rechazo de peticiones no autorizada

```

letl join
  incidentes i1_0
  on i1_0.id=rc1_0.incidente_id
where
  rc1_0.id=?
2026-06-23T00:01:54.225-05:00 INFO 19128 --- [MessageBroker-6] o.s.w.s.c.WebSocketMessageBrokerStats : WebSocketSession[2 current WS(2)-HttpStream(0)-HttpPoll(0),
2026-06-23T00:31:54.228-05:00 INFO 19128 --- [MessageBroker-8] o.s.w.s.c.WebSocketMessageBrokerStats : WebSocketSession[2 current WS(2)-HttpStream(0)-HttpPoll(0),
2026-06-23T00:58:31.371-05:00 ERROR 19128 --- [nio-8081-exec-9] c.b.e.config.GlobalExceptionHandler : Se produjo un error no controlado:
org.springframework.web.HttpRequestMethodNotSupportedException (Create breakpoint) : Request method 'POST' is not supported

```

15. Resultados esperados

La evolución tecnológica planificada para la Central de Bomberos expandirá su cobertura hacia nuevos frentes, estableciendo los siguientes requerimientos no funcionales y metas a futuro:

- **Integración con Hardware IoT:** Se contempla el desarrollo y conexión de dispositivos con sensores especializados de temperatura extrema, concentración de CO2 y humos tóxicos. Estos módulos IoT, operando de manera asíncrona, verificarán de forma autónoma si una vivienda es habitable luego de que un incendio ha sido mitigado. Su dictamen será emitido directamente como un payload hacia el servidor backend y registrado de forma inmutable en la bitácora central de la emergencia.
- **Despliegue Móvil Nativo (APK):** Las cuadrillas en ruta y el personal operativo requerirán una versión portátil de la central. A futuro, se espera envolver el cliente web actual o refactorizarlo para producir un paquete nativo instalable de Android APK, permitiéndoles reportar llegadas al sitio y cierres de incidentes desde los vehículos bomberiles con acceso a la red de telefonía celular.
- **Despliegue Multi-Nodo:** Trasladar la aplicación de un entorno de pruebas a una infraestructura en la nube real. El servicio de Spring Boot se paquetizará en contenedores Docker y se orquestará mediante balanceadores de carga, permitiendo procesar concurrencias masivas a nivel de ciudad.

16. Conclusiones parciales

La segmentación arquitectónica lograda al separar el Servicio de Identidad del Servicio de Negocio Central comprobó que el enfoque de microservicios promueve un código más limpio, escalable y mantenible. Esta modularidad previene cuellos de botella que típicamente aquejan a los sistemas monolíticos heredados.

La integración transversal del protocolo WebSocket para comunicación full-duplex superó categóricamente a la latencia tradicional del protocolo HTTP REST. Proveer al dashboard de la central con notificaciones en vivo transforma una aplicación pasiva en una herramienta táctica crítica para la toma de decisiones rápidas.

El uso estricto del estándar JSON Web Token demostró ser un mecanismo infalible para proteger infraestructuras distribuidas. Al delegar la firma del token a un sistema OAuth y su verificación matemática a filtros sin estado en Java, el sistema alcanzó niveles corporativos de seguridad sin sacrificar recursos de almacenamiento para mantener sesiones de red activas.

17. Referencias bibliográficas

Chen, H., Wang, L., & Zhao, X. (2020). Distributed Transaction Management in Microservices Architecture. *International Conference on Cloud Computing*, 88-95.

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2021). *Distributed Systems: Concepts and Design*. Pearson Education.

Jones, M., & Bradley, J. (2021). Security best practices for OAuth 2.0 and JSON Web Tokens. *IEEE Security & Privacy*, 19(4), 45-53.

Luo, X., & Li, Y. (2022). Real-Time Web Application Architecture Using WebSockets. *Journal of Web Engineering*, 21(3), 112-128.

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems. O'Reilly Media.

Smith, J. (2023). High-Performance Concurrency in Modern Java. O'Reilly Media.

18. Anexos

Link al repositorio: https://github.com/Joseph-Franco692/Proyecto_Emergencias_jf.git